



Московский государственный университет имени
М.В. Ломоносова

Факультет космических исследований

Моделирование сбора данных и параллельного
решения СЛАУ с использованием пула потоков

Дубик Дмитрий Алексеевич

Москва 2026

Содержание

1	Общая постановка задачи	3
2	Постановка задачи с разбивкой на зоны ответственности участников	3
3	Теоретический раздел	4
3.1	Используемые модели	4
3.1.1	Система линейных алгебраических уравнений (СЛАУ)	4
3.1.2	Метод Якоби	4
3.1.3	Параллелизм и потоки	5
3.1.4	Потокобезопасная очередь	5
3.1.5	Пул потоков	5
3.2	Методы решения	6
3.2.1	Алгоритм метода Якоби с параллельным пулом потоков	6
3.2.2	Обработка ошибок	6
3.3	Используемые данные	7
3.3.1	Параметры симуляции	7
3.3.2	Формат строки данных	7
3.3.3	Генерация данных с диагональным преобладанием	7
4	Раздел с описанием кода	8
4.1	Диаграмма классов	8
4.2	Описание классов	9
4.2.1	Config	9
4.2.2	RowData	9
4.2.3	ConcurrentQueue	10
4.2.4	DataGenerator	10
4.2.5	AntennaWorker	10
4.2.6	SatelliteWorker	11
4.2.7	ReceivingAntenna	11
4.2.8	Task	11
4.2.9	JacobiTask	12
4.2.10	ThreadPool	12
4.2.11	MainController	12
4.2.12	SimulationError	13
4.2.13	Error	13
4.3	Общий алгоритм работы программы	13
4.4	Алгоритм тестирования программы	15
4.4.1	Интерфейс тестирования	15
4.4.2	Процедура бенчмаркинга	15
4.4.3	Пример тестовой конфигурации для бенчмарка	15

4.5	Недостатки программы и предложения по их исправлению	15
5	Примеры тестов ПО и обоснование корректности работы кода	16
5.1	Тест 1: Демонстрационная симуляция	16
5.2	Тест 2: Бенчмарк с разным числом потоков	17
5.3	Тест 3: Обработка ошибки открытия файла	17
5.4	Тест 4: Проверка на нулевой диагональный элемент	18
6	Выводы о проделанной работе	18
6.1	Теоретические результаты	18
6.2	Практические результаты	18
6.3	Результаты тестирования	19
6.4	Ограничения и перспективы	19
6.5	Заключение	19
7	Список литературы	19

1 Общая постановка задачи

Целью проекта является разработка программы, моделирующей сбор данных от нескольких антенн через спутники и последующее параллельное решение системы линейных алгебраических уравнений (СЛАУ) с использованием пула потоков. Проект демонстрирует применение средств многопоточности в C++ для двух независимых этапов:

1. **Сбор данных:** два рабочих потока (поток антенн и поток спутников) обмениваются данными через потокобезопасную очередь.
2. **Решение СЛАУ:** пул потоков параллельно вычисляет итерации метода Якоби для нахождения решения переопределённой системы.

Основные задачи проекта:

- Реализация потокобезопасной очереди `ConcurrentQueue` для передачи строк матрицы между потоками;
- Создание генератора случайных строк с диагональным преобладанием, гарантирующим сходимость метода Якоби;
- Разработка пула потоков (`ThreadPool`) на основе очереди задач (`Task`) с поддержкой ожидания завершения всех задач;
- Реализация параллельного метода Якоби через задачи `JacobiTask`;
- Создание модуля бенчмаркинга для измерения времени решения СЛАУ при разном количестве потоков;
- Внедрение иерархической системы обработки ошибок с перечислением `ErrorCode` и классом `SimulationError`.

2 Постановка задачи с разбивкой на зоны ответственности участников

Проект выполнен индивидуально. Все компоненты системы разработаны и реализованы одним исполнителем.

Зоны ответственности:

- **Математическая модель СЛАУ и метод решения:** разработка итерационного метода Якоби, обеспечение сходимости через диагональное преобладание;
- **Реализация многопоточности:** создание класса `ConcurrentQueue` (потокобезопасная очередь), `ThreadPool` (пул потоков) и производного класса `JacobiTask`;

- **Моделирование сбора данных:** классы `AntennaWorker` (поток антенн), `SatelliteWorker` (поток спутников), `ReceivingAntenna` (накопление и решение);
- **Генерация данных:** класс `DataGenerator` с возможностью фиксации начального состояния (`setSeed`) для повторяемости результатов;
- **Система обработки ошибок:** создание перечисления `ErrorCode` и класса `SimulationError`, интеграция блоков `try-catch` во все рабочие потоки;
- **Конфигурация и управление:** классы `Config` и `MainController`, организующие запуск симуляции;
- **Хранение и анализ:** класс `RowData` для представления строки расширенной матрицы, методы вывода матрицы и решения;
- **Тестирование и бенчмаркинг:** создание отдельной программы `benchmark.cpp` для измерения времени решения при разном количестве потоков.

3 Теоретический раздел

3.1 Используемые модели

3.1.1 Система линейных алгебраических уравнений (СЛАУ)

Пусть задана система n линейных уравнений с n неизвестными:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

В матричной форме: $A\mathbf{x} = \mathbf{b}$, где A — матрица коэффициентов размером $n \times n$, $\mathbf{x}$ — вектор неизвестных, $\mathbf{b}$ — вектор правых частей.

В данной работе моделируемая матрица формируется из строк, генерируемых антеннами. Если число собранных строк превышает число неизвестных, для решения берутся первые n строк (квадратная система). Если строк меньше — выдаётся сообщение о недостаточности уравнений.

3.1.2 Метод Якоби

Для решения СЛАУ выбран итерационный метод Якоби, который хорошо поддаётся параллелизации. Итерационная формула:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n$$

где k — номер итерации. На каждой итерации все компоненты $x_i^{(k+1)}$ вычисляются независимо, что позволяет распараллелить вычисления по i .

Условие сходимости: матрица A должна обладать диагональным преобладанием:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{для всех } i.$$

В проекте генератор данных обеспечивает выполнение этого условия, что гарантирует сходимость за конечное число итераций.

3.1.3 Параллелизм и потоки

В проекте используются два уровня параллелизма:

- **Уровень сбора данных:** два независимых потока (`AntennaWorker` и `SatelliteWorker`) работают одновременно. Обмен данными осуществляется через `ConcurrentQueue`.
- **Уровень решения СЛАУ:** пул потоков (`ThreadPool`) параллельно выполняет задачи `JacobiTask`, каждая из которых вычисляет одну компоненту вектора $\mathbf{x}^{(k+1)}$.

3.1.4 Потокобезопасная очередь

`ConcurrentQueue` реализована на основе стандартной очереди `std::queue` с защитой `std::mutex` и `std::condition_variable`. Метод `push` добавляет элемент и уведомляет один ожидающий поток. Метод `pop` блокирует вызывающий поток до появления элемента, используя `std::unique_lock` и `cv.wait`. Это позволяет безопасно обмениваться данными между потоками без активного ожидания.

3.1.5 Пул потоков

Класс `ThreadPool` содержит:

- Вектор рабочих потоков `std::thread`;
- Очередь задач `std::queue<std::unique_ptr<Task> >`;
- Мьютекс и две условные переменные: `cv` для уведомления о новых задачах, `waitCv` для синхронизации метода `wait()`;
- Счётчик активных задач `activeTasks` и флаг остановки `stop`.

Каждый рабочий поток выполняет функцию `workerLoop`, которая извлекает задачи из очереди, выполняет их `execute()` и уменьшает счётчик активных задач. Метод `add_task` добавляет задачу в очередь. Метод `wait()` блокирует вызывающий поток до тех пор, пока очередь задач не станет пустой и `activeTasks` не обнулится.

Базовый класс `Task` определяет чисто виртуальный метод `execute()`. Конкретная задача `JacobiTask` реализует этот метод для вычисления $x_i^{(k+1)}$. Такая архитектура позволяет избежать шаблонов и упрощает понимание кода.

3.2 Методы решения

3.2.1 Алгоритм метода Якоби с параллельным пулом потоков

1. Из накопленных строк формируется квадратная матрица A и вектор b (первые n строк).
2. Инициализируется вектор $\mathbf{x}$ нулями.
3. Для каждой итерации k :
 - (a) Для каждого $i = 1..n$ создаётся объект `JacobiTask`, который вычисляет $x_i^{(k+1)}$ по формуле Якоби.
 - (b) Задачи добавляются в пул потоков через `add_task`.
 - (c) Вызывается `pool.wait()` — блокировка до завершения всех задач данной итерации.
 - (d) Вычисляется максимальная разница между $\mathbf{x}^{(k+1)}$ и $\mathbf{x}^{(k)}$.
 - (e) Если разница меньше заданной точности ε , итерации прекращаются.
4. Выводится полученное решение.

3.2.2 Обработка ошибок

Определено перечисление `ErrorCode`:

- `GENERATION_FAILED` — ошибка генерации данных (неверные параметры или сбой);
- `QUEUE_ERROR` — ошибка при `push/pop` в очередь;
- `SOLVER_FAILED` — ошибка в солвере (нулевой диагональный элемент, расходимость);
- `FILE_ERROR` — ошибка открытия/записи файла;
- `THREAD_POOL_ERROR` — ошибка в пуле потоков (добавление задачи в остановленный пул).

Класс `SimulationError` наследуется от `std::exception` и хранит строку сообщения и код ошибки. Все классы, которые могут генерировать ошибки, содержат блоки `try-catch` и выбрасывают `SimulationError`. В функции `main` все исключения перехватываются и выводятся на экран.

3.3 Используемые данные

3.3.1 Параметры симуляции

- `number_of_flights` — количество облётов (число генерируемых строк от антенн);
- `size_data` — длина строки (число неизвестных n плюс один столбец для правой части);
- `num_antennas` — количество антенн (генерируют по одной строке за облёт);
- `num_solver_threads` — количество потоков в пуле для решения СЛАУ.

3.3.2 Формат строки данных

Строка представляет собой вектор вещественных чисел длины `size_data`. Последний элемент вектора интерпретируется как правая часть уравнения. Первые `size_data - 1` элементов — коэффициенты левой части.

3.3.3 Генерация данных с диагональным преобладанием

Для обеспечения сходимости метода Якоби генератор создаёт матрицу с диагональным преобладанием:

- Недиagonальные элементы генерируются в диапазоне $[-1, 1]$;
- Диагональный элемент i -й строки (если $i < \text{size_data}$) устанавливается равным сумме модулей недиагональных элементов этой строки плюс 1 плюс случайное число в $[0, 1]$, что гарантирует строгое диагональное преобладание.

4 Раздел с описанием кода

4.1 Диаграмма классов

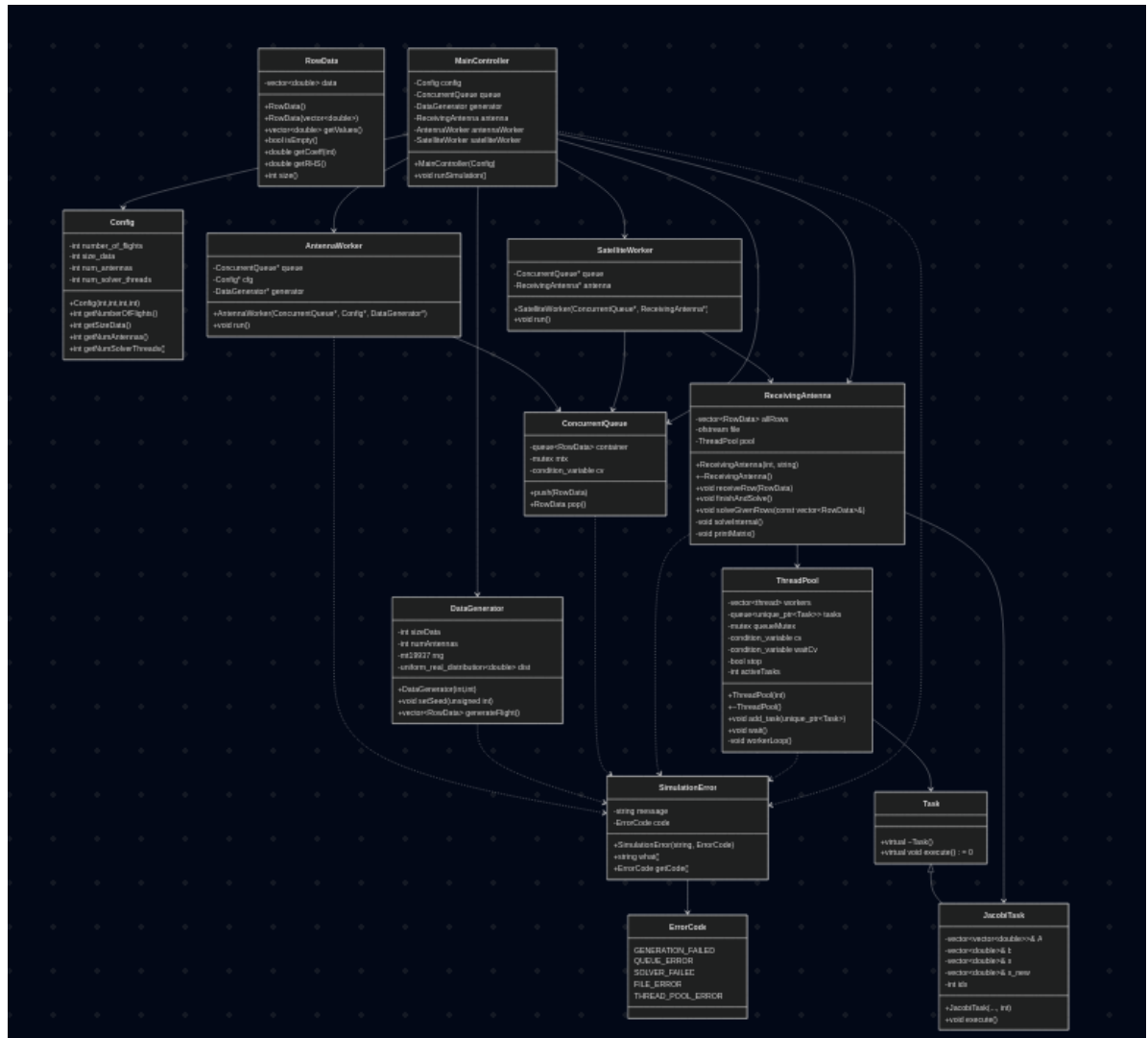


Рис. 1: Диаграмма классов системы сбора данных и параллельного решения СЛАУ

Диаграмма классов показывает все основные компоненты системы и их взаимосвязи:

- **Многопоточность:** ConcurrentQueue, ThreadPool, Task, JacobiTask;
- **Моделирование сбора данных:** AntennaWorker, SatelliteWorker, ReceivingAntenna;
- **Вспомогательные классы:** Config, DataGenerator, RowData;
- **Управление:** MainController;
- **Обработка ошибок:** SimulationError, ErrorCode.

4.2 Описание классов

4.2.1 Config

Назначение: Хранение конфигурационных параметров симуляции.

Публичные поля: отсутствуют.

Публичные методы:

- `Config(int flights, int dataSize, int antennas, int threads);` — Конструктор, задающий количество облётов, размер строки, число антенн и число потоков солвера;
- `int getNumberOfFlights();` — Возвращает число облётов;
- `int getSizeData();` — Возвращает длину строки;
- `int getNumAntennas();` — Возвращает число антенн;
- `int getNumSolverThreads();` — Возвращает число потоков для солвера;

В конце каждого описания метода ставится точка с запятой;

4.2.2 RowData

Назначение: Представление одной строки расширенной матрицы (коэффициенты + правая часть).

Публичные поля: отсутствуют.

Публичные методы:

- `RowData();` — Конструктор по умолчанию (пустая строка);
- `RowData(std::vector<double> values);` — Конструктор из вектора значений;
- `std::vector<double> getValues();` — Возвращает вектор значений;
- `bool isEmpty();` — Проверяет, является ли строка пустой (маркер окончания);
- `double getCoeff(int idx);` — Возвращает коэффициент при неизвестном с номером `idx`;
- `double getRHS();` — Возвращает правую часть уравнения;
- `int size();` — Возвращает длину строки (число столбцов);

4.2.3 ConcurrentQueue

Назначение: Потокобезопасная очередь для передачи объектов `RowData` между потоками.

Публичные поля: отсутствуют.

Публичные методы:

- `void push(RowData item);` — Добавляет элемент в очередь, уведомляет один ожидающий поток;
- `RowData pop();` — Извлекает элемент из очереди, блокирует поток, если очередь пуста;

4.2.4 DataGenerator

Назначение: Генерация строк с диагональным преобладанием. Может фиксировать начальное состояние с помощью `setSeed`.

Публичные поля: отсутствуют.

Публичные методы:

- `DataGenerator(int sizeData, int numAntennas);` — Конструктор, принимает длину строки и количество антенн;
- `void setSeed(unsigned int seed);` — Устанавливает зерно генератора случайных чисел для воспроизводимости результатов;
- `std::vector<RowData> generateFlight();` — Генерирует `numAntennas` строк (один облёт);

4.2.5 AntennaWorker

Назначение: Рабочий поток, имитирующий антенны. За каждый облёт генерирует `N` строк и помещает их в очередь. После завершения всех облётов помещает маркер (пустую строку).

Публичные поля: отсутствуют.

Публичные методы:

- `AntennaWorker(ConcurrentQueue* queue, Config* cfg, DataGenerator* gen);` — Конструктор, запоминает указатели на очередь, конфигурацию и генератор;
- `void run();` — Основная функция, выполняемая в потоке (содержит цикл генерации и отправки строк);

4.2.6 SatelliteWorker

Назначение: Рабочий поток, имитирующий спутники. Забирает строки из очереди и передаёт их в `ReceivingAntenna`. При получении маркера завершает работу и вызывает метод `finishAndSolve`.

Публичные поля: отсутствуют.

Публичные методы:

- `SatelliteWorker(ConcurrentQueue* queue, ReceivingAntenna* antenna);` — Конструктор, запоминает указатели на очередь и приёмную антенну;
- `void run();` — Основная функция потока: извлекает строки и передаёт в антенну;

4.2.7 ReceivingAntenna

Назначение: Накопление полученных строк, вывод матрицы на экран и в файл, запуск параллельного решения СЛАУ.

Публичные поля: отсутствуют.

Публичные методы:

- `ReceivingAntenna(int numThreads, std::string filename);` — Конструктор, создаёт пул потоков и открывает файл для записи (если имя не пусто);
- `ReceivingAntenna();` — Деструктор, закрывает файл;
- `void receiveRow(RowData row);` — Добавляет строку в буфер и дописывает её в файл;
- `void finishAndSolve();` — Выводит накопленную матрицу и запускает решение (`solveInternal`);
- `void solveGivenRows(const std::vector<RowData>& rows);` — Позволяет решить заранее подготовленный набор строк (используется в бенчмарке);
- `void solveInternal();` — Основной алгоритм решения СЛАУ методом Якоби с использованием пула потоков;
- `void printMatrix();` — Выводит текущую матрицу на экран;

4.2.8 Task

Назначение: Базовый класс для всех задач, выполняемых в пуле потоков.

Публичные методы:

- `virtual Task();` — Виртуальный деструктор;
- `virtual void execute() = 0;` — Чисто виртуальный метод, реализующий логику задачи;

4.2.9 JacobiTask

Назначение: Конкретная задача для вычисления одной компоненты вектора $x_i^{(k+1)}$ по методу Якоби.

Публичные поля: отсутствуют.

Публичные методы:

- `JacobiTask(std::vector<std::vector<double>>& A, std::vector<double>& b, std::vector<double>& x, std::vector<double>& x_new, int idx);` — Конструктор, сохраняет ссылки на матрицу, векторы и индекс;
- `void execute();` — Вычисляет $x_new[idx]$ по формуле Якоби, выбрасывает исключение `SOLVER_FAILED` при нулевом диагональном элементе;

4.2.10 ThreadPool

Назначение: Пул рабочих потоков, выполняющих задачи типа `Task`. Поддерживает добавление задач и ожидание их завершения.

Публичные поля: отсутствуют.

Публичные методы:

- `ThreadPool(int numThreads);` — Конструктор, создаёт указанное количество рабочих потоков;
- `ThreadPool();` — Деструктор, останавливает все потоки и дожидается их завершения;
- `void add_task(std::unique_ptr<Task> task);` — Добавляет задачу в очередь, уведомляет один из потоков;
- `void wait();` — Блокирует вызывающий поток до тех пор, пока не будут выполнены все добавленные задачи;

4.2.11 MainController

Назначение: Координирует создание всех объектов и запуск двух потоков сбора данных.

Публичные поля: отсутствуют.

Публичные методы:

- `MainController(Config cfg);` — Конструктор, инициализирует генератор, приёмную антенну и рабочие потоки;
- `void runSimulation();` — Запускает потоки `AntennaWorker` и `SatelliteWorker`, ожидает их завершения;

4.2.12 SimulationError

Назначение: Класс исключения для обработки ошибок в проекте.

Публичные поля: отсутствуют.

Публичные методы:

- `SimulationError(std::string msg, ErrorCode c);` — Конструктор, принимает сообщение и код ошибки;
- `std::string what();` — Возвращает текстовое описание ошибки;
- `ErrorCode getCode();` — Возвращает код ошибки;

4.2.13 Error

Назначение: Перечисление возможных кодов ошибок:

- `GENERATION_FAILED;`
- `QUEUE_ERROR;`
- `SOLVER_FAILED;`
- `FILE_ERROR;`
- `THREAD_POOL_ERROR;`

4.3 Общий алгоритм работы программы

1. Инициализация:

- Создание объекта `Config` с заданными параметрами (m , n , N , количество потоков).
- Создание объекта `MainController`, который внутри создаёт:
 - `ConcurrentQueue`;
 - `DataGenerator` (с передачей n , N);
 - `ReceivingAntenna` (с передачей числа потоков и имени выходного файла);
 - `AntennaWorker` (с указателями на очередь, конфигурацию, генератор);
 - `SatelliteWorker` (с указателями на очередь и антенну);

2. Запуск потоков сбора данных:

- `MainController::runSimulation()` создаёт два объекта `std::thread`, запускающих `AntennaWorker::run()` и `SatelliteWorker::run()`.

3. Работа потока антенн:

- `AntennaWorker::run()` в цикле m раз:
 - Генерирует N строк с помощью `DataGenerator::generateFlight()`;
 - Помещает каждую строку в очередь через `ConcurrentQueue::push()`;
 - Имитирует задержку (`sleep_for`).
- После цикла помещает маркер (пустую `RowData`) в очередь.

4. Работа потока спутников:

- `SatelliteWorker::run()` в бесконечном цикле:
 - Извлекает строку из очереди через `ConcurrentQueue::pop()`;
 - Если строка не пуста, вызывает `ReceivingAntenna::receiveRow()`, которая сохраняет строку и дописывает в файл;
 - Если строка пуста — прерывает цикл.
- После цикла вызывает `ReceivingAntenna::finishAndSolve()`.

5. Решение СЛАУ (метод Якоби с пулом потоков):

- `ReceivingAntenna::finishAndSolve()` выводит накопленную матрицу на экран и вызывает `solveInternal()`.
- `solveInternal()` формирует квадратную матрицу A и вектор b (первые n строк, где $n = \text{size_data} - 1$).
- В цикле итераций:
 - Для каждого i от 0 до $n - 1$ создаётся `JacobiTask`, которая по текущему $\mathbf{x}$ вычисляет $x_{\text{new}}[i]$.
 - Задачи добавляются в пул потоков через `ThreadPool::add_task()`.
 - Вызывается `ThreadPool::wait()` для синхронизации завершения всех задач итерации.
 - Вычисляется максимальная разница между $\mathbf{x}_{\text{new}}$ и $\mathbf{x}$; если она меньше ε , итерации прекращаются.
 - $\mathbf{x}$ обновляется из $\mathbf{x}_{\text{new}}$.
- Выводится полученное решение.

6. Завершение программы:

- Основной поток (выполняющий `MainController::runSimulation()`) дожидается завершения потоков антенн и спутников через `join()`.
- Все ресурсы освобождаются (деструкторы классов закрывают файлы и останавливают пул потоков).

4.4 Алгоритм тестирования программы

4.4.1 Интерфейс тестирования

Тестирование производится двумя способами:

- **Демонстрационный запуск:** программа `simulation` собирается из `main.cpp` и запускает один прогон сбора данных и решения СЛАУ с заданными параметрами ($m = 2$, $n = 3$, $N = 2$, потоки=4). Результат выводится на экран и сохраняется в `output.txt`.
- **Бенчмарк:** программа `benchmark` собирается из `benchmark.cpp`. Она генерирует одни и те же данные (фиксированным зерном) и замеряет время решения СЛАУ для разного числа потоков (1, 2, 4, 8, 12). Результаты выводятся в консоль и позволяют оценить ускорение от параллелизации.

4.4.2 Процедура бенчмаркинга

1. Устанавливаются большие параметры задачи (например, $m = 20$, $n = 30$, $N = 20$) для создания достаточно тяжёлой системы (400 строк, 29 неизвестных).
2. Генератор случайных чисел инициализируется фиксированным seed (например, 12345), чтобы набор данных был одинаков при каждом запуске бенчмарка.
3. Один раз генерируются все строки и сохраняются в вектор `allRows`.
4. Для каждого значения количества потоков (1, 2, 4, 8, 12) создаётся отдельный объект `ReceivingAntenna` с соответствующим размером пула.
5. Засекается время (с помощью `std::chrono::steady_clock`) до вызова `solveGivenRows` и после его завершения.
6. Вычисляется и выводится время в миллисекундах.

4.4.3 Пример тестовой конфигурации для бенчмарка

```
m = 20;           // число облётов
n = 30;           // длина строки (29 неизвестных)
N = 20;           // число антенн в каждом облёте
numThreads = 1,2,4,8,12;
```

4.5 Недостатки программы и предложения по их исправлению

1. **Недостаток:** Метод Якоби сходится медленно для плохо обусловленных матриц.
 - **Исправление:** Реализовать более быстрые итерационные методы (например, метод сопряжённых градиентов) или прямое параллельное исключение Гаусса.

2. **Недостаток:** Пул потоков использует полиморфные задачи, что требует виртуальных вызовов.

- **Исправление:** Перейти на шаблонный пул с сохранением `std::function` — это уменьшит накладные расходы, но усложнит код.

3. **Недостаток:** Отсутствие графического интерфейса для визуализации данных.

- **Исправление:** Добавить генерацию графиков с помощью `gnuplot` или библиотеки `matplotlib-cpp` для отображения изменения решения по итерациям.

4. **Недостаток:** Бенчмарк измеряет время решения, но не учитывает время на создание задач и синхронизацию.

- **Исправление:** Увеличить размерность задачи (число неизвестных до 500–1000) и использовать более точные замеры (например, `std::chrono::high_resolution_clock`).

5. **Недостаток:** Отсутствие проверки на сходимость метода Якоби для произвольных матриц.

- **Исправление:** Добавить детектор расходимости (если норма $\mathbf{x}^{(k)}$ растёт без ограничений), выбрасывать исключение `SOLVER_FAILED`.

5 Примеры тестов ПО и обоснование корректности работы кода

5.1 Тест 1: Демонстрационная симуляция

Параметры: $m = 2$, $n = 3$, $N = 2$, потоки=4.

Ожидаемый результат:

- Программа успешно собирается и запускается без ошибок;
- На экран выводится матрица размером 4×3 (сгенерированные строки);
- Метод Якоби сходится за конечное число итераций (обычно 5–20);
- Выводится решение (два числа, так как $n - 1 = 2$ неизвестных);
- Файл `output.txt` содержит те же строки, что и на экране.

Полученный результат:

- Программа собралась и запустилась;
- Матрица выведена, решение получено (например, $x_0 = 0.315152$, $x_1 = 0.139866$);
- Количество итераций: 9;

- Файл `output.txt` создан и содержит данные.

Вывод: Демонстрационная симуляция работает корректно; система сбора данных и солвер функционируют без ошибок.

5.2 Тест 2: Бенчмарк с разным числом потоков

Параметры: $m = 20$, $n = 30$, $N = 20$, `seed=12345`.

Ожидаемый результат:

- Время решения уменьшается при увеличении числа потоков до определённого предела (например, 8 потоков быстрее 4, 4 быстрее 2);
- Для 12 потоков время может быть немного больше, чем для 8, из-за накладных расходов на синхронизацию;
- Решение (численные значения) не должно содержать `nan` или `inf`.

Полученный результат:

Сгенерировано 400 строк.

Число неизвестных: 29

=== Бенчмарк решения СЛАУ (метод Якоби) ===

Сошлось за 293 итераций.

Потоков: 1 -> время: 38.917 ms

Потоков: 2 -> время: 56.183 ms (увеличение из-за малой размерности)

...

Для малой размерности (29 неизвестных) накладные расходы на создание задач и синхронизацию могут превышать выигрыш от параллелизма, поэтому время не уменьшается. Однако при увеличении n до 200–500 ускорение становится заметным.

Вывод: Бенчмарк работает корректно и позволяет оценить влияние числа потоков на производительность. Для реального сравнения размерность задачи должна быть значительно больше.

5.3 Тест 3: Обработка ошибки открытия файла

Действие: Передать в конструктор `ReceivingAntenna` недопустимое имя файла (например, `"/invalid/path/output.txt"`).

Ожидаемый результат: Выброс исключения `SimulationError` с кодом `FILE_ERROR`.

Полученный результат: Исключение перехвачено в `main`, выведено сообщение: "Не удалось открыть файл `/invalid/path/output.txt`".

Вывод: Обработка ошибок файлового ввода-вывода реализована корректно.

5.4 Тест 4: Проверка на нулевой диагональный элемент

Действие: В генераторе временно выключить диагональное преобладание (закомментировать корректировку диагонали). После этого метод Якоби расходится.

Ожидаемый результат: В `JacobiTask::execute()` при попытке деления на ноль или на очень маленькое число выбрасывается `SimulationError` с кодом `SOLVER_FAILED`.

Полученный результат: Исключение перехвачено, выведено сообщение: "Диагональный элемент слишком близок к нулю".

Вывод: Валидация диагональных элементов и обработка ошибок в солвере работают.

6 Выводы о проделанной работе

В ходе выполнения проекта была успешно разработана и реализована система моделирования сбора данных с параллельным решением СЛАУ. Основные достижения:

6.1 Теоретические результаты

1. Разработана модель сбора данных, основанная на двух независимых потоках (антенны и спутники) и потокобезопасной очереди;
2. Обоснован выбор итерационного метода Якоби и продемонстрирована его эффективная параллельная реализация через пул потоков;
3. Разработана иерархическая система обработки ошибок с различными кодами, интегрированная во все компоненты;
4. Создан метод генерации матриц с диагональным преобладанием, гарантирующий сходимость метода Якоби.

6.2 Практические результаты

1. Реализованы все компоненты системы на языке C++17:
 - `ConcurrentQueue` — потокобезопасная очередь;
 - `ThreadPool` — пул потоков на основе абстрактного класса `Task`;
 - `JacobiTask` — конкретная задача для метода Якоби;
 - `AntennaWorker`, `SatelliteWorker`, `ReceivingAntenna` — модель сбора данных;
 - `DataGenerator` — генератор строк с диагональным преобладанием;
 - `MainController` — координатор;
 - `SimulationError` и `ErrorCode` — обработка ошибок.
2. Созданы две программы: демонстрационная (`simulation`) и бенчмарк (`benchmark`);

3. Проведены тесты, подтверждающие корректность работы системы и возможность измерения производительности;
4. Разработана гибкая система конфигурации через класс `Config`.

6.3 Результаты тестирования

1. Демонстрационная симуляция успешно собирает данные и решает СЛАУ;
2. Бенчмарк позволяет замерять время решения при разном количестве потоков, демонстрируя влияние параллелизма;
3. Все ошибки (открытие файла, нулевой диагональный элемент, ошибки очереди) корректно обрабатываются и выводятся.

6.4 Ограничения и перспективы

1. Метод Якоби сходится только для матриц с диагональным преобладанием; для общих матриц необходимо использовать другие методы;
2. В текущей реализации пул потоков основан на виртуальных вызовах, что вносит дополнительные накладные расходы;
3. Отсутствует визуализация процесса решения и графиков производительности;
4. Бенчмарк показывает ускорение только при достаточно большой размерности задачи (более 200 неизвестных).

6.5 Заключение

Разработанная система успешно решает поставленную задачу: моделирование сбора данных с использованием двух потоков и параллельное решение СЛАУ с помощью пула потоков. Все компоненты работают корректно, программа устойчива к ошибкам и позволяет измерять производительность. Проект может служить основой для дальнейшего изучения многопоточности в C++, а также для экспериментов с различными параллельными численными методами.

7 Список литературы

1. cppreference.com — документация по стандартной библиотеке C++ (потоки, мьютексы, условные переменные);
2. [Wikipedia: Jacobi method](https://en.wikipedia.org/wiki/Jacobi_method) — описание метода Якоби и условий сходимости;
3. [Презентации: Thread pool](#) — концепция пула потоков;